Experiment – 7

Title of the Experiment: Data Type Conversion and Transformation Across CSV, JSON, and XML Datasets

Aim of the Experiment: To convert data between different types such as string-to-integer, string-to-float, and string-to-date, and to apply data viewing and cleaning methods on CSV, JSON, and XML datasets.

Software Required: Python 3.x, Jupyter Notebook, NumPy

Algorithm:

Test Cases:

Tracing of Code:

Expected Output:

PART A: LOAD AND INSPECT DATA

```
import pandas as pd
import numpy as np

print("===== PART A: LOAD AND INSPECT DATA =====")

# Load datasets
df_csv = pd.read_csv("employees.csv")
df_json = pd.read_json("employees.json")
df_xml = pd.read_xml("employees.xml", parser="etree")

# View data
print(df_csv.head())
print(df_csv.info())
```

OutPut:

```
===== PART A: LOAD AND INSPECT DATA =====
   emp_id     name  salary joining_date
0     100     Raju   50000   15-06-2021
1     101     Siva  100000  2020/010/20
2     102      Ram   90000   10-04-2022
3     103   Lohith   25000   15-12-2021
4     104  Murugan  150000   15-07-2021
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   emp_id        8 non-null      int64
 1   name          8 non-null      object
 2   salary        8 non-null      int64
 3   joining_date  8 non-null      object
dtypes: int64(2), object(2)
memory usage: 388.0+ bytes
None
```

PART B: STRING TO NUMERIC CONVERSION

```
print("\n===== PART B: STRING TO NUMERIC CONVERSION =====")
# Convert emp_id to integer
df_csv['emp_id'] = df_csv['emp_id'].astype(int)
df_json['emp_id'] = df_json['emp_id'].astype(int)
df_xml['emp_id'] = df_xml['emp_id'].astype(int)
```

```
===== PART B: STRING TO NUMERIC CONVERSION =====
```

```
# Convert salary to float
df_csv['salary'] = pd.to_numeric(df_csv['salary'], errors='coerce')
df_json['salary'] = pd.to_numeric(df_json['salary'], errors='coerce')
df_xml['salary'] = pd.to_numeric(df_xml['salary'], errors='coerce')

# Handle missing salaries
df_csv['salary'] = df_csv['salary'].fillna(df_csv['salary'].mean())
df_json['salary'] = df_json['salary'].fillna(df_json['salary'].mean())
df_xml['salary'] = df_xml['salary'].fillna(df_xml['salary'].mean())

print("Cleaned Salary Data:")
print(df_csv[['emp_id', 'salary']])
```

OutPut:

```
Cleaned Salary Data:
   emp_id  salary
0     100   50000
1     101  100000
2     102   90000
3     103   25000
4     104  150000
5     105  100000
6     106  200000
7     107  200000
```

PART C: STRING TO DATE CONVERSION

```
print("\n===== PART C: STRING TO DATE CONVERSION =====")
# Convert joining_date to datetime
df_csv['joining_date'] = pd.to_datetime(df_csv['joining_date'], errors='coerce')
df_json['joining_date'] = pd.to_datetime(df_json['joining_date'], errors='coerce')
df_xml['joining_date'] = pd.to_datetime(df_xml['joining_date'], errors='coerce')
```

```
===== PART C: STRING TO DATE CONVERSION =====
```

```
# Extract year and month
df_csv['joining_year'] = df_csv['joining_date'].dt.year
df_json['joining_month'] = df_json['joining_date'].dt.month
df_xml['joining_year'] = df_xml['joining_date'].dt.year

print("CSV Dates:")
print(df_csv[['emp_id', 'joining_date', 'joining_year']])
```

OutPut:

```
CSV Dates:
   emp_id joining_date  joining_year
0     100   2021-06-15        2021.0
1     101          NaT           NaN
2     102   2022-04-10        2022.0
3     103   2021-12-15        2021.0
4     104   2021-07-15        2021.0
5     105   2020-08-20        2020.0
6     106   2022-10-10        2022.0
7     107   2021-08-15        2021.0
```

```python
print("\n===== PART B: JSON DATA HANDLING =====")
# Read JSON
df_json = pd.read_json("students.json")
print(df_json)
```

FINAL COMBINED DATA

```python
print("\n===== FINAL COMBINED DATA =====")

combined_df = pd.concat([df_csv, df_json, df_xml], ignore_index=True)
print(combined_df)
print(combined_df.dtypes)
```

OutPut:

```
===== FINAL COMBINED DATA =====
    emp_id     name  salary joining_date  joining_year  joining_month
0      100     Raju   50000   2021-06-15        2021.0            NaN
1      101     Siva  100000          NaT           NaN            NaN
2      102      Ram   90000   2022-04-10        2022.0            NaN
3      103   Lohith   25000   2021-12-15        2021.0            NaN
4      104  Murugan  150000   2021-07-15        2021.0            NaN
5      105   Govind  100000   2020-08-20        2020.0            NaN
6      106    Durga  200000   2022-10-10        2022.0            NaN
7      107  Lakshmi  200000   2021-08-15        2021.0            NaN
8      100     Raju   50000   2021-06-15           NaN            6.0
9      101     Siva  100000          NaT           NaN            NaN
10     102      Ram   90000   2022-04-10           NaN            4.0
11     103   Lohith   25000   2021-12-15           NaN           12.0
12     104  Murugan  150000   2021-07-15           NaN            7.0
13     105   Govind  100000   2020-08-20           NaN            8.0
14     106    Durga  200000   2022-10-10           NaN           10.0
15     107  Lakshmi  200000   2021-08-15           NaN            8.0
16     100     Raju   50000   2021-06-15        2021.0            NaN
17     101     Siva  100000          NaT           NaN            NaN
18     102      Ram   90000   2022-04-10        2022.0            NaN
19     103   Lohith   25000   2021-12-15        2021.0            NaN
20     104  Murugan  150000   2021-07-15        2021.0            NaN
21     105   Govind  100000   2020-08-20        2020.0            NaN
22     106    Durga  200000   2022-10-10        2022.0            NaN
23     107  Lakshmi  200000   2021-08-15        2021.0            NaN
emp_id                     int64
name                      object
salary                     int64
joining_date     datetime64[ns]
joining_year            float64
joining_month           float64
dtype: object
```